Documentation technique de l’analyse et l’évolution de la base de données Millenuit

Alexis, Charles, Hugo

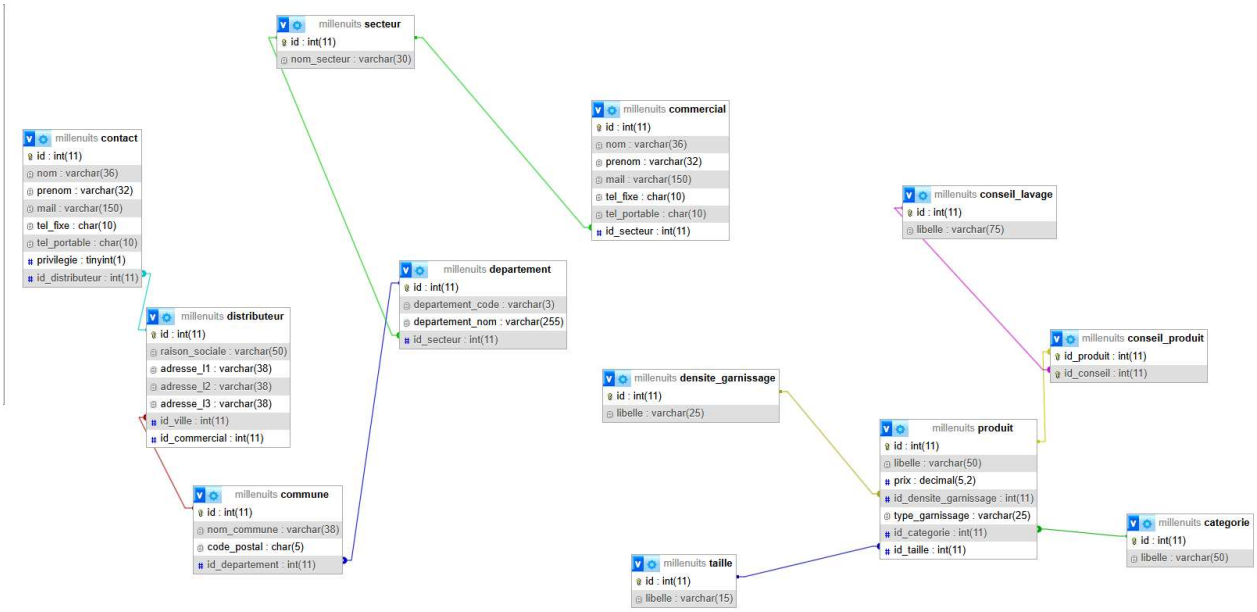
Sommaire

Prérequis :....3
Création de la Base de données et importation des éléments :....3
Correction des Anomalies de la base de données :....4
Type Produit :....4
Dimension :....4
Prix produit :....5
Mail Commercial :....5
Téléphone commercial :....5
Téléphone Contact :....6
Id_Distributeur :....7
Evolution de la base de données :....7
Gestion à l'accès de l'application :....10

Intoduction :

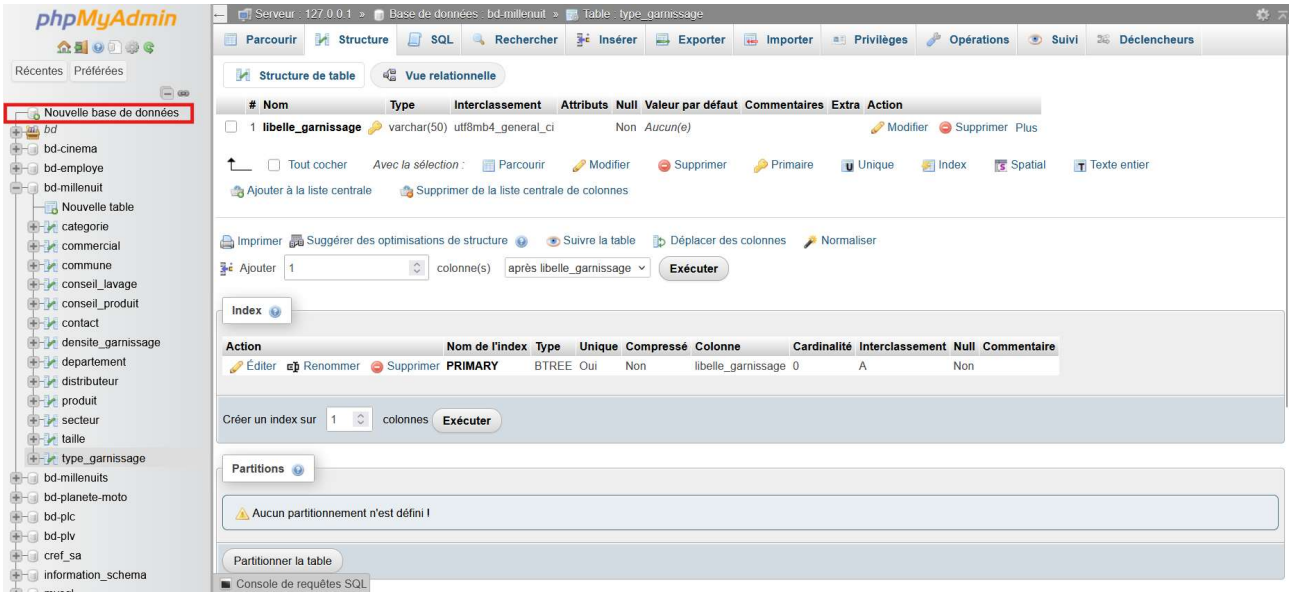
L’entreprise Millenuits nous a demander d’analyser et de faire l’évolution de leur base de données car elle n’était pas conforme à leurs besoins. De plus, il fallait corriger les anomalies et mettre à jour le jeu d’essai. Ainsi que compléter la base de données afin de prendre en compte les évolutions et alimenter les nouvelles tables avec un jeu d’essai.

Base de données de Millenuits au départ :



Création de la Base de données et importation des éléments :

Dans un premier temps nous avons créer la base de données « MilleNuit » puis nous avons importé les tables et les données dans la base de données.


phpMyAdmin
Récentes Préférées
Nouvelle base de données
bd
bd-cinema
bd-employe
bd-millenuit
Nouvelle table
categorie
commercial
commune
conseil_lavage
conseil_produit
contact
densite_garnissage
departement
distributeur
produit
secteur
taille
type_garnissage
bd-millenuits
bd-planete-moto
bd-plc
bd-plv
cref_sa
information_schema
Serveur : 127.0.0.1 » Base de données : bd-millenuit » Table : type_garnissage
Parcourir Structure SQL Rechercher Insérer Exporter Importer Privilèges Opérations Suivi Déclencheurs
Structure de table Vue relationnelle
Nom Type Interclassement Attributs Null Valeur par défaut Commentaires Extra Action
1 libelle_garnissage varchar(50) utf8mb4_general_ci Non Aucun(e) Modifier Supprimer Plus
Tout cocher Avec la sélection : Parcourir Modifier Supprimer Primaire Unique Index Spatial Texte entier
Ajouter à la liste centrale Supprimer de la liste centrale de colonnes
Imprimer Suggérer des optimisations de structure Suivre la table Déplacer des colonnes Normaliser
Ajouter 1 colonne(s) après libelle_garnissage Exécuter
Index
Action Nom de l'index Type Unique Compressé Colonne Cardinalité Interclassement Null Commentaire
Éditer Renommer Supprimer PRIMARY BTREE Oui Non libelle_garnissage 0 A Non
Créer un index sur 1 colonnes Exécuter
Partitions
Aucun partitionnement n'est défini !
Partitionner la table
Console de requêtes SQL

Correction des Anomalies de la base de données :

Type Produit :

Afin de mettre le type de garnissage en liste déroulante dans la table produit, il nous à fallu créer une nouvelle table nommer type_garnissage avec id_garnissage et libelle_garnissage

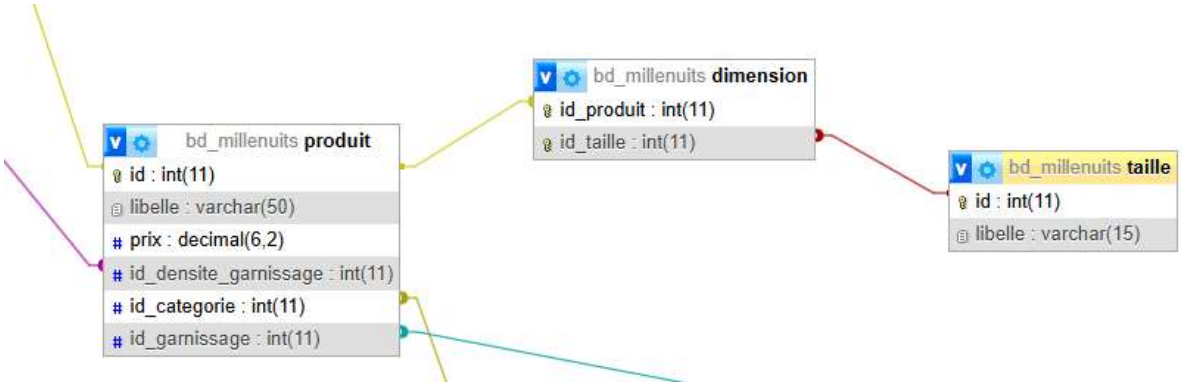


Nous avons créer une ligne dans produit nommer id_garnissage fessant référence à id_garnissage de type_garnissage.



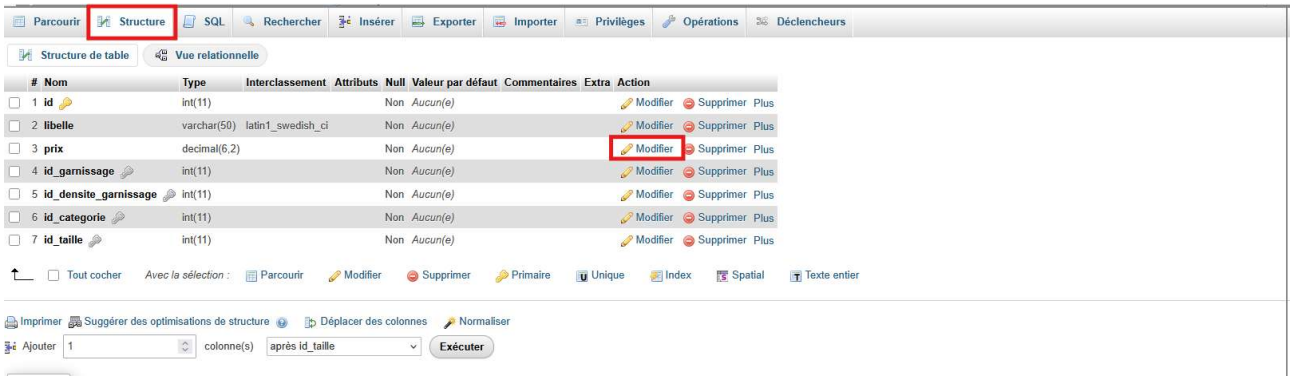
Dimension :

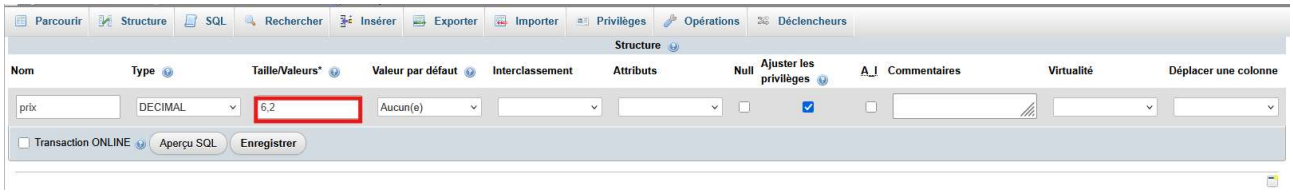
Dans le but que plusieurs produits puissent avoir plusieurs dimensions, nous avons créer une table nommer dimension qui à pour but de rentrer un id de produit et un id de taille pour qu'un produit puisse avoir plusieurs taille.



Prix produit :

pour permettre de mettre le prix des produits à 4 chiffres (1198) il faut modifier la taille permise dans la ligne prix et la changer de 5,2 à 6,2. Cela signifie qu'il pourra y avoir jusu'à 6 chiffre devant la virgule et 2 derrière la virgule.





Le passage d'une précision de 5 à 6 chiffres permet d'augmenter la valeur maximale autorisée. Alors qu'une limite à 5 chiffres restreignait les montants à 999,99, une précision de 6 permet désormais d'atteindre 9 999,99, couvrant ainsi les prix dépassant le millier d'euros.

Mail Commercial :

Afin d'automatiser la génération des adresses e-mail à partir du prénom et du nom des contacts, nous avons utilisé la commande UPDATE. Celle-ci modifie la table existante en concaténant les informations d'identité avec le nom de domaine de l'entreprise.
« UPDATE commercial

SET mail = LOWER (CONCAT(prenom,'.', nom, '@millenuits.com')) »

Téléphone commercial :

Modifier le numéro de téléphone portable de chaque commercial avec la requête suivante :

```
UPDATE commercial SET tel_portable = '0647253895';
```

Donc dans la table commercial :

id	nom	prenom	mail	tel_fixe	tel_portable	id_secteur
1	De	Philippe	philippe.de@millenuit.com	NULL	0647253895	1
2	Villechalane	Louis	louis.villechalane@millenuit.com	NULL	0642195379	1
3	Andre	David	david.andre@millenuit.com	NULL	0643187359	2
4	Bedos	Christian	christian.bedos@millenuit.com	NULL	0642193865	1

Téléphone Contact :

Afin de garantir l'intégrité des données, les numéros de téléphone non renseignés (précédemment saisis sous la forme "9999999999") ont été convertis en valeur NULL. Pour ce faire, une requête UPDATE a été exécutée, et la structure de la table a été modifiée pour définir NULL comme valeur par défaut pour ce champ.

UPDATE commercial SET tel_portable = NULL WHERE tel_portable = '9999999999';

tel_fixe	tel_portable	privilegie
0965691359	0625698957	
0965692718	0625698958	
NULL	0665748985	
0925681710	NULL	
0965691312	NULL	
0965691360	NULL	
0965691357	NULL	
0925681741	NULL	
0925681789	NULL	
NULL	0665748970	
NULL	0665748945	
0925681743	NULL	
0925681744	NULL	
NULL	0765748981	
NULL	0665748965	
0945892563	NULL	
0945892526	NULL	
0945892529	NULL	
0945892528	NULL	
0945892524	0765748954	

Id_Distributeur :

L'erreur liée à l'identifiant id_commercial provenait de l'absence de relation (clé étrangère) entre les tables commercial et distributeur.

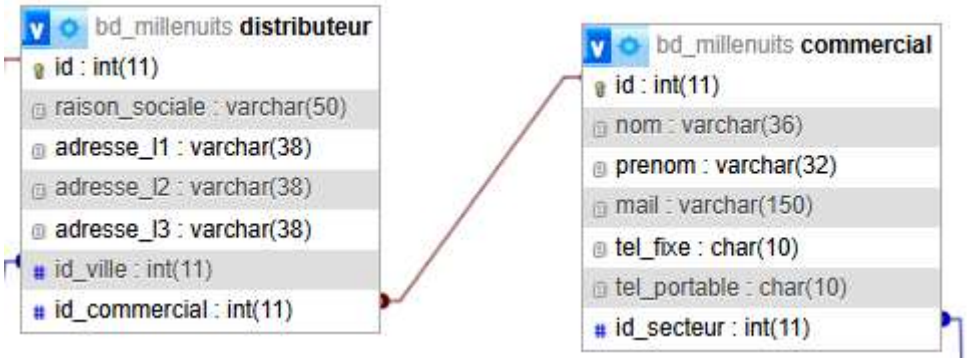
Afin de remédier à ce problème, nous avons d'abord corrigé la valeur erronée dans la table distributeur à l'aide de la commande suivante :

UPDATE distributeur SET id_commercial = '1' WHERE distributeur.id = 1;

Résultat :

id	raison_sociale	adresse_l1	adresse_l2	adresse_l3	id_ville	id_commercial
1	PLACE DE LA LITERIE	8 allée du Haut Poitou	Les Portes du Futur	*NULL*	34215	1

Puis nous avons ajouter la contrainte de clé étrangère entre distributeur et commercial



Evolution de la base de données :

L'évolution de la base de données avait pour objectif d'enrichir le suivi commercial et de structurer la gestion des visites auprès des distributeurs.

La table Motif : Création de la table motif : pour catégoriser les raisons des rendez-vous

```
CREATE TABLE motif( id_motif int NOT NULL, libelle_motif varchar(50), CONSTRAINT pk_mot
if PRIMARY KEY (id_motif) );
```



La table visite :

Création de la table visite : pour centraliser les données clés

```
CREATE TABLE visite( id_visite int NOT NULL, date_visite date NOT NULL, bilan_visite tex
t NOT NULL, personne_rencontree varchar(50) NOT NULL, coefConfiance varchar(36) NOT NUL
L, id_motif int NOT NULL, id_commercial int NOT NULL, id_distributeur int NOT NULL, id_c
ontact INT not null, CONSTRAINT pk_visite PRIMARY KEY (id_visite) );
```



La table produit presenté :

Afin de savoir quels produits sont mis en avant lors des rendez-vous, nous avons créé une table de liaison produit_presente. Celle-ci permet d'associer un ou plusieurs produits à une visite spécifique, offrant ainsi un historique précis des présentations effectuées par les commerciaux.

```
CREATE TABLE produit_presente( id_produit int NOT NULL, id_visite int NOT NULL, CONSTRA
INT pk_produit_presente PRIMARY KEY (id_produit, id_visite) );
```



Faire les liens entre les tables avec les clés :

table visite :

-avec la table motif :

```
ALTER TABLE visite ADD CONSTRAINT fk_visite_motif FOREIGN KEY (id_motif) REFERENCES motif(id_motif);
```

-avec la table commerciale :

```
ALTER TABLE visite ADD CONSTRAINT fk_visite_commercial FOREIGN KEY (id_commercial) REFERENCES commercial(id);
```

-avec la table distributeur :

```
ALTER TABLE visite ADD CONSTRAINT fk_visite_distributeur FOREIGN KEY (id_distributeur) REFERENCES distributeur(id);
```

-avec la table contact :

```
ALTER TABLE visite ADD CONSTRAINT fk_visite_contact FOREIGN KEY (id_contact) REFERENCES contact(id);
```

table produit présenté :

-avec la table produit :

```
ALTER TABLE produit_presente ADD CONSTRAINT fk_produit_presente_produit FOREIGN KEY (id_produit) REFERENCES produit(id);
```

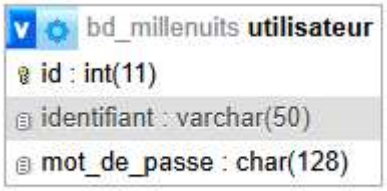
-avec la table visite :

```
ALTER TABLE produit_presente ADD CONSTRAINT fk_produit_presente_visite FOREIGN KEY (id_visite) REFERENCES visite(id_visite);
```

Gestion à l’accès de l’application :

Création d’une table utilisateur :

```
CREATE TABLE utilisateur( id int AUTO_INCREMENT, identifiant varchar(50) NOT NULL, mot_de_passe char(128) NOT NULL, CONSTRAINT pk_utilisateur PRIMARY KEY (id) );
```



avec l’identifiant qui sera composé de la première lettre du prénom + le nom.

Et le mot_de_passe qui est haché :

```
CREATE TRIGGER cree_utilisateur
BEFORE INSERT ON utilisateur
FOR EACH ROW
BEGIN
    SET NEW.mot_de_passe=SHA2(NEW.mot_de_passe, 512);
END //
```

test :

Le mot de passe rentré en clair à bien été hashé.

id	identifiant	mot_de_passe
1	jHytde	662cf42c6e2d0e475fd4366699e5c1dc708a8693610006a8a3...

Schéma finale :

